

Early problem identification: If issues are identified early they are much easier and cost-effective to fix. Defects found later are always more expensive to fix.

In-depth problem identification: Testing cannot identify coding related issues as it cannot cover every possible execution path. But as static code analysis checks your code, it gives you an in-depth report of where the code can cause potential problems. Testing cannot find unreachable code but static code analysis can identify it.

Gives you high quality code: Use of static code analysis in daily software development will result in high quality code that complies with industry standards. In the long run, the code will be easy to maintain too.

Drawbacks of static code analysis

Static code analysis does have a few drawbacks.

Can report false positives or false negatives: Consider the code given in the example below. Here, `func()` is not yet executed and we know nothing about it. So, we don't know what value of number will be returned.

```
int square(void)
{
    int number;
    if(func())
    {
        number = 0;
    }
    else
    {
        number = 7;
    }
    return (number*number);
}
```

As the value returned by `func()` in the code is undecided, the static code analysis may report an issue that may not be an issue at all (false positive), or it may not report an issue which should be reported (false negative).

Sometimes rules are not enforced statically: While setting standards there might be some rules which could be referenced from different sources such as standards like CERT, MISRA, and OWASP, which may not necessarily be applicable to all your code.

Developer's intentions are not understood: As static code analysis works based on rule sets and standards, it is not capable of determining the function's expectations, which might not match some of the rules defined in the standards. This means it cannot determine that the function fundamentally does not do what is expected as compliant to standards.

Here's a quick look at the best static code analysis tools available in the market.

Static code analysis tools that support multiple languages

There are many tools available in the market to detect issues related to code. There are tools for analysing multiple languages and also to analyse a single language. But we will focus on the top five tools for static code analysis which analyse multiple languages.

SonarQube

SonarQube is the most widely used open source Web based static analysis tool for continuously inspecting the code quality and security of the entire code, as well as guiding development teams to solve these issues quickly during code reviews. SonarQube finds different types of issues, vulnerabilities, bugs and code smells. It also keeps track of duplications, unit test results and code coverage in a single dashboard. SonarQube is mainly used for automated static code analysis with continuous integration. It can be integrated with multiple tools including different IDEs, Jenkins, Azure DevOps, and many more.

What makes SonarQube different is that it provides quality metrics about the code, which will help the developer to take the right decision. It translates these nondescript values to real business

values such as risk, code coverage and technical debt.

Languages supported: 27 programming languages, which are ABAP, Apex, C, C#, C++, CSS, COBOL, Flex, Go, HTML, Java (Java SE, Java EE, JSP), JavaScript, Kotlin, Objective-C, PHP, PLI, PL/SQL, Python, Ruby, Swift, Scala, TypeScript, TSQL, VB.NET, Visual Basic 6, XML

Features:

- Multi-language support
- Security, maintainability and reliability analysis
- Quality gate checks
- Dashboards and portfolios for audit purpose
- Integrates with IDE, continuous integration servers (Jenkins, Azure DevOps, and many more)
- Good community support

Limitations:

- Supports only a few IDEs
- No support for multi-target platform builds

Pricing: Community edition is free and open source. Licensed versions are available.

Codacy

Codacy is a static code analysis tool that allows a programmer to tackle technical debt and improve code quality. It automatically analyses code quality on every commit and pull request. It maintains the code by blocking pull requests, which ultimately saves time in code review. It checks code quality and keeps track of your technical debt for more than 30 programming languages. It can be integrated within your development workflow seamlessly. You can enforce your code quality standards, enforce high-security practices, and save time in code review.

Languages supported: 30+ languages including Elixir, Go, Java (Java EE, Java SE, JSP), JavaScript, JSON, Kotlin, Python, Ruby, Scala, Swift, TypeScript, and more